

R PROGRAMMING LAB

PRACTICAL NO 8

Name: Aditi Gawari	
Branch: CMPN	Division: A
Roll No: 23102A0025	Batch: 1
Date of Practical: 06/10/2026	Date of Submission: 09/10/2026

Lab Problem Statement 8: High-Performance Big Data Analytics Using R

1. Aim

To develop a high-performance and scalable data analytics solution in R using the NYC Taxi Trip dataset and evaluate the computational efficiency of different R programming approaches.

2. Objectives

- Import, clean, and preprocess a large taxi-trip dataset.
- Perform efficient data manipulation using `data.table`.
- Analyze hourly, daily, and monthly taxi demand, fares, revenue, routes, and payment preferences.
- Compare `apply()`, `lapply()`, and `purrr::map()` with vectorized operations and `data.table`.
- Compare sequential and parallel processing using `foreach` and `doParallel`.
- Benchmark execution time and calculate parallel speedup.
- Create visualizations using `ggplot2`.
- Recommend suitable approaches based on performance, memory efficiency, readability, and scalability.

3. Dataset

Dataset Name: NYC Taxi & Limousine Commission (TLC) – Yellow Taxi Trip Records

<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

A. Import the dataset and inspect its structure

Inspecting Taxi Dataset:

```
> head(taxi)
  vendorID tpep_pickup_datetime tpep_dropoff_datetime
    <int>          <POSct>          <POSct>
1:      2 2026-01-01 06:24:04 2026-01-01 06:29:37
2:      1 2026-01-01 06:04:04 2026-01-01 06:09:47
3:      1 2026-01-01 06:27:06 2026-01-01 06:35:59
4:      2 2026-01-01 05:45:22 2026-01-01 06:28:10
5:      2 2026-01-01 05:57:13 2026-01-01 06:10:43
6:      2 2026-01-01 06:17:11 2026-01-01 06:30:47
  passenger_count trip_distance RatecodeID store_and_fwd_flag
    <int>          <num>          <int>          <char>
1:         1          0.97           1             N
2:         0          0.90           1             N
3:         0          1.40           1             N
4:         4          5.58           1             N
5:         0          2.16           1             N
6:         2          2.33           1             N
  PULocationID DOLocationID payment_type fare_amount extra mta_tax
    <int>          <int>          <int>          <num> <num> <num>
1:      239      238           1           7.2 1.00 0.5
2:      163      162           2           7.9 4.25 0.5
3:       43      237           1          10.7 4.25 0.5
4:      142      209           1          38.7 1.00 0.5
5:       88      144           1          13.5 1.00 0.5
6:      144      137           1          14.2 1.00 0.5
  tip_amount tolls_amount improvement_surcharge total_amount
    <num>          <num>          <num>          <num>
1:     3.66         0           1           15.86
2:     0.00         0           1           13.65
3:     2.50         0           1           18.95
4:    11.11         0           1           55.56
```

```
> str(taxi)
Classes 'data.table' and 'data.frame': 3724889 obs. of 20 variables:
 $ vendorID      : int  2 1 1 2 2 2 1 1 2 2 ...
 $ tpep_pickup_datetime : POSIXct, format: "2026-01-01 06:24:04" ...
 $ tpep_dropoff_datetime: POSIXct, format: "2026-01-01 06:29:37" ...
 $ passenger_count  : int  1 0 0 4 0 2 1 0 1 3 ...
 $ trip_distance    : num  0.97 0.9 1.4 5.58 2.16 2.33 1.3 2.9 5.34
 1.83 ...
 $ RatecodeID      : int  1 1 1 1 1 1 1 1 1 1 ...
 $ store_and_fwd_flag : chr  "N" "N" "N" "N" ...
 $ PULocationID    : int  239 163 43 142 88 144 142 50 161 237 ...
 $ DOLocationID    : int  238 162 237 209 144 137 50 234 45 263 ...
 $ payment_type     : int  1 2 1 1 1 1 2 1 1 1 ...
 $ fare_amount      : num  7.2 7.9 10.7 38.7 13.5 14.2 11.4 22.6 37.3
 10.7 ...
 $ extra           : num  1 4.25 4.25 1 1 1 4.25 4.25 1 1 ...
 $ mta_tax         : num  0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5
 ...
 $ tip_amount       : num  3.66 0 2.5 11.11 3.85 ...
 $ tolls_amount     : num  0 0 0 0 0 0 0 0 0 0 ...
 $ improvement_surcharge: num  1 1 1 1 1 1 1 1 1 1 ...
 $ total_amount     : num  15.9 13.6 18.9 55.6 23.1 ...
 $ congestion_surcharge : num  2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5 2.5
 ...
 $ Airport_fee      : num  0 0 0 0 0 0 0 0 0 0 ...
 $ cbd_congestion_fee : num  0 0.75 0.75 0.75 0.75 0.75 0.75 0.75 0.75 0.75
 0 ...
 - attr(*, ".internal.selfref")=<pointer: 0x00000196333fecf0>
```

```
> dim(taxi)
[1] 3724889      20
> names(taxi)
 [1] "VendorID"           "tpep_pickup_datetime"
 [3] "tpep_dropoff_datetime" "passenger_count"
 [5] "trip_distance"      "RatecodeID"
 [7] "store_and_fwd_flag" "PULocationID"
 [9] "DOLocationID"       "payment_type"
[11] "fare_amount"        "extra"
[13] "mta_tax"           "tip_amount"
[15] "tolls_amount"      "improvement_surcharge"
[17] "total_amount"      "congestion_surcharge"
[19] "Airport_fee"       "cbd_congestion_fee"
> summary(taxi)
  vendorID      tpep_pickup_datetime
Min.   :1.000   Min.   :2026-01-01 05:27:29
1st Qu.:2.000   1st Qu.:2026-01-09 23:21:59
Median :2.000   Median :2026-01-17 02:50:56
Mean   :1.874   Mean   :2026-01-17 07:14:52
3rd Qu.:2.000   3rd Qu.:2026-01-24 12:55:11
Max.   :7.000   Max.   :2026-02-01 06:15:01

 tpep_dropoff_datetime  passenger_count  trip_distance
Min.   :2026-01-01 05:27:32  Min.   :0.000   Min.   :0.000e+00
1st Qu.:2026-01-09 23:38:47  1st Qu.:1.000   1st Qu.:1.000e+00
Median :2026-01-17 03:06:14  Median :1.000   Median :1.810e+00
Mean   :2026-01-17 07:32:03  Mean   :1.256   Mean   :6.456e+00
3rd Qu.:2026-01-24 13:10:22  3rd Qu.:1.000   3rd Qu.:3.730e+00
Max.   :2026-02-02 05:05:31  Max.   :9.000   Max.   :2.691e+05
      NAS :1088058

> format(object.size(taxi), units = "MB")
[1] "483.1 Mb"
```

Checking the lookup table

```
> head(zones)
  LocationID      Borough      Zone service_zone
    <int>      <char>      <char>      <char>
1:         1      EWR      Newark Airport      EWR
2:         2    Queens      Jamaica Bay      Boro Zone
3:         3    Bronx Allerton/Pelham Gardens      Boro Zone
4:         4    Manhattan      Alphabet City      Yellow Zone
5:         5 Staten Island      Arden Heights      Boro Zone
6:         6 Staten Island Arrochar/Fort Wadsworth      Boro Zone
> |
```

B. Data cleaning and preprocessing

```
> print(missing_counts)
  vendorID tpep_pickup_datetime tpep_dropoff_datetime
    <int>      <int>      <int>
1:         0              0              0
  passenger_count trip_distance RatecodeID store_and_fwd_flag
    <int>      <int>      <int>      <int>
1:    1088058              0    1088058    1088058
  PULocationID DOLocationID payment_type fare_amount extra mta_tax
    <int>      <int>      <int>      <int> <int> <int>
1:         0              0              0         0         0         0
  tip_amount tolls_amount improvement_surcharge total_amount
    <int>      <int>      <int>      <int>
1:         0              0              0         0
  congestion_surcharge Airport_fee cbd_congestion_fee
    <int>      <int>      <int>
1:    1088058    1088058         0
> |
```

Cleaned dataset:

```
> dim(taxi)
[1] 3562058      20
> summary(taxi)
 vendorID      tpep_pickup_datetime
Min.   :1.000   Min.   :2025-12-31 23:57:29
1st Qu.:2.000   1st Qu.:2026-01-09 17:42:24
Median :2.000   Median :2026-01-16 20:58:55
Mean   :1.869   Mean   :2026-01-17 00:56:10
3rd Qu.:2.000   3rd Qu.:2026-01-24 03:42:22
Max.   :7.000   Max.   :2026-02-01 00:45:01

 tpep_dropoff_datetime      passenger_count      trip_distance
Min.   :2025-12-31 23:57:32   Min.   :0.000   Min.   :1.000e-02
1st Qu.:2026-01-09 18:00:21   1st Qu.:1.000   1st Qu.:1.090e+00
Median :2026-01-16 21:14:20   Median :1.000   Median :1.900e+00
Mean   :2026-01-17 01:13:26   Mean   :1.254   Mean   :6.705e+00
3rd Qu.:2026-01-24 03:55:52   3rd Qu.:1.000   3rd Qu.:3.860e+00
Max.   :2026-02-01 23:35:31   Max.   :8.000   Max.   :2.691e+05
      NAS :995312

 RatecodeID      store_and_fwd_flag      PULocationID
Min.   : 1.00    Length :3562058   Min.   : 1.0
1st Qu.: 1.00    N.unique : 2      1st Qu.:114.0
Median : 1.00    N.blank  : 0      Median :161.0
Mean   : 5.29    Min.nchar: 1      Mean   :161.7
3rd Qu.: 1.00    Max.nchar: 1      3rd Qu.:233.0
Max.   :99.00    NAS      : 995312   Max.   :265.0
      NAS :995312

 DOLocationID      payment_type      fare_amount      extra
Min.   : 1.0   Min.   :0.000   Min.   : 0.00   Min.   : -0.750
1st Qu.:107.0   1st Qu.:0.000   1st Qu.: 10.00   1st Qu.: 0.000
Median :162.0   Median :1.000   Median : 15.60   Median : 0.000
```

C. Extract Temporal Attributes

```
> head(
+   taxi[,.(
+     tpep_pickup_datetime,
+     pickup_hour,
+     pickup_day,
+     pickup_weekday,
+     pickup_month,
+     trip_duration_min
+   )]
+ )
 tpep_pickup_datetime pickup_hour pickup_day pickup_weekday
      <POSct>          <int>      <int>      <ord>
1: 2026-01-01 00:54:04          0          1    Thursday
2: 2026-01-01 00:34:04          0          1    Thursday
3: 2026-01-01 00:57:06          0          1    Thursday
4: 2026-01-01 00:15:22          0          1    Thursday
5: 2026-01-01 00:27:13          0          1    Thursday
6: 2026-01-01 00:47:11          0          1    Thursday
 pickup_month trip_duration_min
      <ord>          <num>
1:    January      5.550000
2:    January      5.716667
3:    January      8.883333
4:    January     42.800000
5:    January     13.500000
6:    January     13.600000
Error in .rs.exprMutatesPackageLibrary(part) :
  argument "part" is missing, with no default
> |
```

D. Join the Taxi Zone Lookup Table

Join results

```
> head(
+   taxi[, .(
+     pickup_zone,
+     pickup_borough,
+     dropoff_zone,
+     dropoff_borough
+   )]
+ )
```

	pickup_zone <char>	pickup_borough <char>	dropoff_zone <char>
1:	Upper West Side South	Manhattan	Upper West Side North
2:	Midtown North	Manhattan	Midtown East
3:	Central Park	Manhattan	Upper East Side South
4:	Lincoln Square East	Manhattan	Seaport
5:	Financial District South	Manhattan	Little Italy/NoLiTa
6:	Little Italy/NoLiTa	Manhattan	Kips Bay

```
dropoff_borough
<char>
1: Manhattan
2: Manhattan
3: Manhattan
4: Manhattan
5: Manhattan
6: Manhattan
```

E. Transportation data analytics using data.table

a. Number of taxi trips by hour

```
> hourly_demand <- taxi[
+   , .(total_trips = .N),
+   by = pickup_hour
+ ][order(pickup_hour)]
>
> print(hourly_demand)
```

	pickup_hour <int>	total_trips <int>
1:	0	109369
2:	1	75924
3:	2	52641
4:	3	37562
5:	4	27740
6:	5	31942
7:	6	62383
8:	7	111593
9:	8	145804
10:	9	154001
11:	10	158231
12:	11	168194
13:	12	183425
14:	13	191392
15:	14	202830
16:	15	216585
17:	16	206431
18:	17	224225
19:	18	232874

b. Taxi demand by day of the week

```
> weekday_demand <- taxi[
+   , .(total_trips = .N),
+   by = pickup_weekday
+ ][order(pickup_weekday)]
>
> print(weekday_demand)
  pickup_weekday total_trips
      <ord>         <int>
1:      Monday      358979
2:     Tuesday      458911
3:   Wednesday      480467
4:    Thursday      622473
5:      Friday      617954
6:    Saturday      658124
7:      Sunday      365150
```

c. Monthly taxi demand

```
> monthly_demand <- taxi[
+   , .(total_trips = .N),
+   by = .(pickup_month_num, pickup_month)
+ ][order(pickup_month_num)]
>
> print(monthly_demand)
  pickup_month_num pickup_month total_trips
      <num>         <ord>         <int>
1:           1      January      3562052
2:           2     February           1
3:          12      December           5
```

d. Average fare and total revenue

```
> fare_analysis <- taxi[
+   , .(
+     total_trips = .N,
+     average_fare = mean(fare_amount, na.rm = TRUE),
+     total_revenue = sum(total_amount, na.rm = TRUE),
+     average_trip_distance = mean(
+       trip_distance,
+       na.rm = TRUE
+     )
+   ),
+   by = pickup_hour
+ ][order(pickup_hour)]
>
> print(fare_analysis)
  pickup_hour total_trips average_fare total_revenue
      <int>         <int>         <num>         <num>
1:          0      109369      21.96915      3299769.9
2:          1       75924      21.63954      2226765.8
3:          2       52641      20.41177      1464514.0
4:          3       37562      21.00532      1063607.9
5:          4       27740      24.19035       879442.6
6:          5       31942      26.95001      1109541.6
7:          6       62383      25.85453      2067232.6
8:          7      111593      23.93571      3497038.8
9:          8      145804      22.58721      4384393.5
10:         9      154001      20.69379      4363667.5
11:        10      158231      19.97637      4402816.0
12:        11      168194      19.69657      4633364.3
13:        12      183425      20.05251      5146990.4
14:        13      191392      20.33925      5445549.7
15:        14      200000      21.36117      6033500.0
```

e. Most frequently travelled routes

```
> top_routes <- taxi[
+   !is.na(pickup_zone) & !is.na(dropoff_zone),
+   .(
+     total_trips = .N
+   ),
+   by = .(
+     pickup_zone,
+     dropoff_zone
+   )
+ ][order(-total_trips)]
>
> print(head(top_routes, 10))
```

	pickup_zone <char>	dropoff_zone <char>	total_trips <int>
1:	Upper East Side South	Upper East Side North	23836
2:	Upper East Side North	Upper East Side South	20668
3:	Upper East Side North	Upper East Side North	16465
4:	Upper East Side South	Upper East Side South	15207
5:	Midtown Center	Upper East Side South	10158
6:	Upper East Side South	Midtown Center	9351
7:	Lincoln Square East	Upper West Side South	8926
8:	Upper West Side South	Upper West Side North	8743
9:	Upper West Side South	Lincoln Square East	8425
10:	Lenox Hill West	Upper East Side North	8409

f. Highest-revenue routes

```
> print(head(top_revenue_routes, 10))
```

	pickup_zone <char>	dropoff_zone <char>	total_trips <int>
1:	JFK Airport	Outside of NYC	7146
2:	JFK Airport	Times Sq/Theatre District	4902
3:	Upper East Side South	Upper East Side North	23836
4:	LaGuardia Airport	Times Sq/Theatre District	4678
5:	Upper East Side North	Upper East Side South	20668
6:	JFK Airport	Clinton East	2586
7:	JFK Airport	Midtown South	2465
8:	Upper East Side North	Upper East Side North	16465
9:	LaGuardia Airport	Midtown Center	2934
10:	Times Sq/Theatre District	LaGuardia Airport	2783

	total_revenue <num>	average_revenue_per_trip <num>
1:	879258.5	123.04205
2:	476810.7	97.26859
3:	383166.2	16.07510
4:	375229.8	80.21158
5:	351816.4	17.02228
6:	249569.1	96.50778
7:	242751.2	98.47921

g. Relationship between trip distance and fare

```
> print(distance_fare)
      distance_group average_fare total_trips
      <fctr>         <num>         <int>
1:      0-1 miles      9.566069      802158
2:      1-3 miles     15.193094     1621596
3:      5-10 miles    34.994816     408902
4:      3-5 miles     25.071031     455192
5:     10-20 miles    57.794946     243893
6:      20+ miles     90.866659      30317
>
> # Correlation between distance and fare
> cor(
+   taxi$trip_distance,
+   taxi$fare_amount,
+   use = "complete.obs"
+ )
[1] 0.00761774
> |
```

h. Payment preferences

```
> payment_analysis <- taxi[
+   , .(
+     total_trips = .N,
+     total_revenue = sum(total_amount, na.rm = TRUE)
+   ),
+   by = payment_type
+ ][order(-total_trips)]
>
> print(payment_analysis)
      payment_type total_trips total_revenue
      <int>         <int>         <num>
1:             1      2231447     65605534.2
2:             0       995312     31363053.2
3:             2       297883      7469257.8
4:             4        27899      831821.8
5:             3         9517      241304.3
> |
```

Demand by borough

```
> borough_demand <- taxi[
+   !is.na(pickup_borough),
+   .(
+     total_trips = .N,
+     average_fare = mean(fare_amount, na.rm = TRUE),
+     total_revenue = sum(total_amount, na.rm = TRUE)
+   ),
+   by = pickup_borough
+ ][order(-total_trips)]
>
> print(borough_demand)
      pickup_borough total_trips average_fare total_revenue
      <char>         <int>         <num>         <num>
1:      Manhattan    3051535     17.47988     77875758.56
2:      Queens       323258      48.56761     20962761.10
3:      Brooklyn    146585      30.84995      5169746.77
4:      Bronx        35313      32.67232     1281756.67
5:      unknown      4088       20.40583     122895.55
6:      N/A          707       74.86564      61291.12
7:      Staten Island 455       41.70369     23750.55
8:      EWR          117       95.51880     13010.93
> |
```

F. Functional programming versus optimized programming

Method A — Functional programming using lapply()

```
> functional_result <- lapply(
+   hour_groups,
+   function(x) mean(x, na.rm = TRUE)
+ )
>
> print(functional_result)
$`0`
[1] 21.76791

$`1`
[1] 21.7352

$`2`
[1] 20.92997

$`3`
[1] 20.27935

$`4`
[1] 24.91476

$`5`
[1] 26.75243

$`6`
[1] 25.9882

$`7`
[1] 23.87624

$`8`
[1] 22.80524
```

Method B — Functional programming using purrr::map()

```
> map_result <- purrr::map_dbl(
+   hour_groups,
+   ~ mean(.x, na.rm = TRUE)
+ )
>
> print(map_result)
      0      1      2      3      4      5      6
21.76791 21.73520 20.92997 20.27935 24.91476 26.75243 25.98820
      7      8      9     10     11     12     13
23.87624 22.80524 20.82439 20.39470 19.16735 20.07952 20.29204
     14     15     16     17     18     19     20
21.26467 21.59096 20.93681 19.84819 19.58526 19.73879 20.94109
     21     22     23
20.87406 21.23333 21.86640
>
```

Method C — Vectorized programming

```
> vectorized_result <- tapply(
+   taxi_sample$fare_amount,
+   taxi_sample$pickup_hour,
+   mean,
+   na.rm = TRUE
+ )
>
> print(vectorized_result)
```

	0	1	2	3	4	5	6
21.76791	21.73520	20.92997	20.27935	24.91476	26.75243	25.98820	
	7	8	9	10	11	12	13
23.87624	22.80524	20.82439	20.39470	19.16735	20.07952	20.29204	
	14	15	16	17	18	19	20
21.26467	21.59096	20.93681	19.84819	19.58526	19.73879	20.94109	
	21	22	23				
20.87406	21.23333	21.86640					

```
> |
```

Method D — data.table

```
> datatable_result <- taxi_sample[
+   , .(average_fare = mean(fare_amount, na.rm = TRUE)),
+   by = pickup_hour
+ ]
>
> print(datatable_result)
```

	pickup_hour	average_fare
	<int>	<num>
1:	14	21.26467
2:	17	19.84819
3:	22	21.23333
4:	16	20.93681
5:	20	20.94109
6:	1	21.73520
7:	9	20.82439
8:	21	20.87406
9:	3	20.27935
10:	18	19.58526
11:	23	21.86640
12:	7	23.87624
13:	19	19.73879
14:	13	20.29204
15:	12	20.07952
16:	4	24.91476
17:	15	21.59096
18:	10	20.39470

G. Sequential versus parallel processing

```
> # Divide the data into four partitions
> number_of_partitions <- 4L
>
> partition_id <- (
+   seq_len(nrow(taxi)) - 1L
+ ) %% number_of_partitions + 1L
>
> taxi_parts <- lapply(
+   seq_len(number_of_partitions),
+   function(i) taxi[partition_id == i]
+ )
>
> # Check partition sizes
> sapply(taxi_parts, nrow)
```

[1]	890515	890515	890514	890514
-----	--------	--------	--------	--------

```
> |
```

Implement Sequential Processing

```
> print(sequential_time)
  user  system elapsed
0.09   0.04   0.09
> print(sequential_summary)
  pickup_hour total_trips
      <int>      <int>
1:         0      109369
2:         1       75924
3:         2       52641
4:         3       37562
5:         4       27740
6:         5       31942
7:         6       62383
8:         7      111593
9:         8      145804
10:        9      154001
11:       10      158231
12:       11      168194
13:       12      183425
14:       13      191392
15:       14      202830
16:       15      216585
17:       16      206431
18:       17      224225
19:       18      232874
20:       19      211623
21:       20      208614
22:       21      208390
23:       22      199989
```

Implement parallel processing using foreach and doParallel

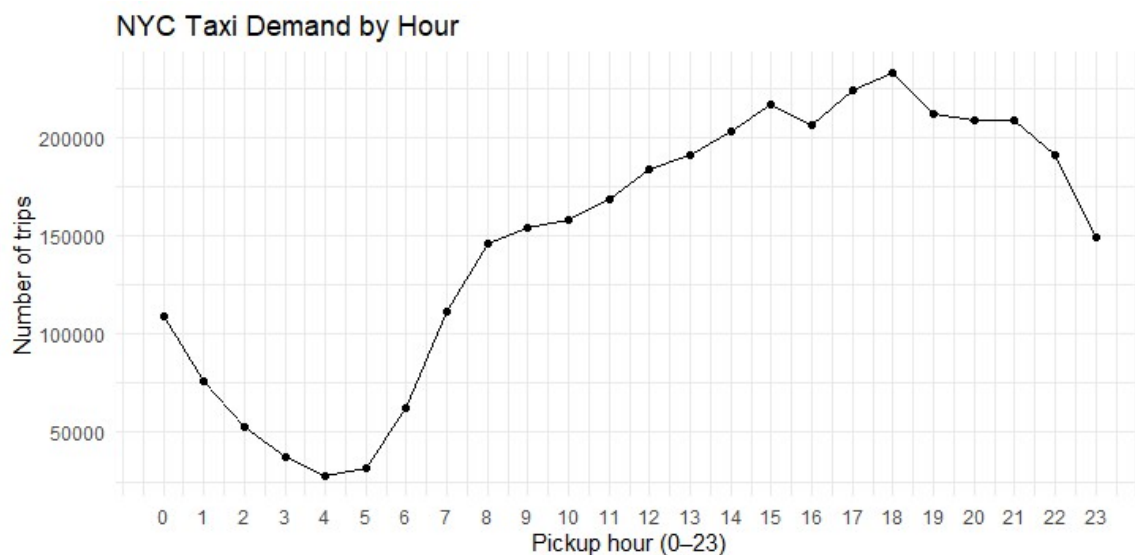
```
> print(parallel_time)
  user  system elapsed
3.36   2.77  12.08
> print(parallel_summary)
  pickup_hour total_trips
      <int>      <int>
1:         0      109369
2:         1       75924
3:         2       52641
4:         3       37562
5:         4       27740
6:         5       31942
7:         6       62383
8:         7      111593
9:         8      145804
10:        9      154001
11:       10      158231
12:       11      168194
13:       12      183425
14:       13      191392
15:       14      202830
16:       15      216585
17:       16      206431
18:       17      224225
19:       18      232874
```

Verify results and calculate speedup

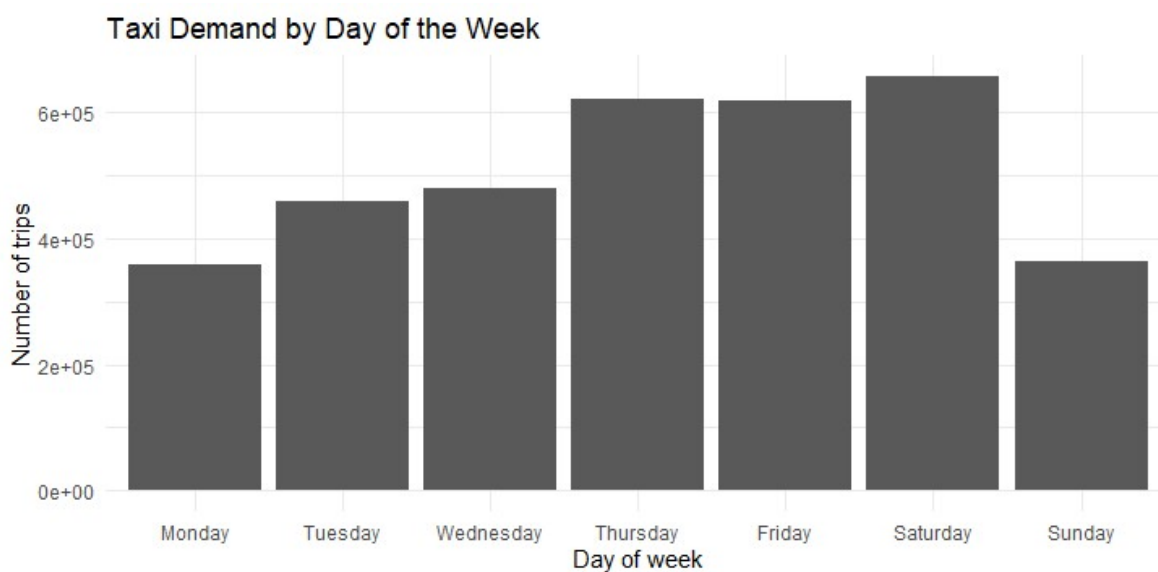
```
> cat("Sequential elapsed time:",
+     sequential_seconds, "seconds\n")
Sequential elapsed time: 0.09 seconds
>
> cat("Parallel elapsed time:",
+     parallel_seconds, "seconds\n")
Parallel elapsed time: 12.08 seconds
>
> cat("Speedup:",
+     round(speedup, 3), "\n")
Speedup: 0.007
> |
```

H. Data visualization using ggplot2

1. Number of taxis per hour



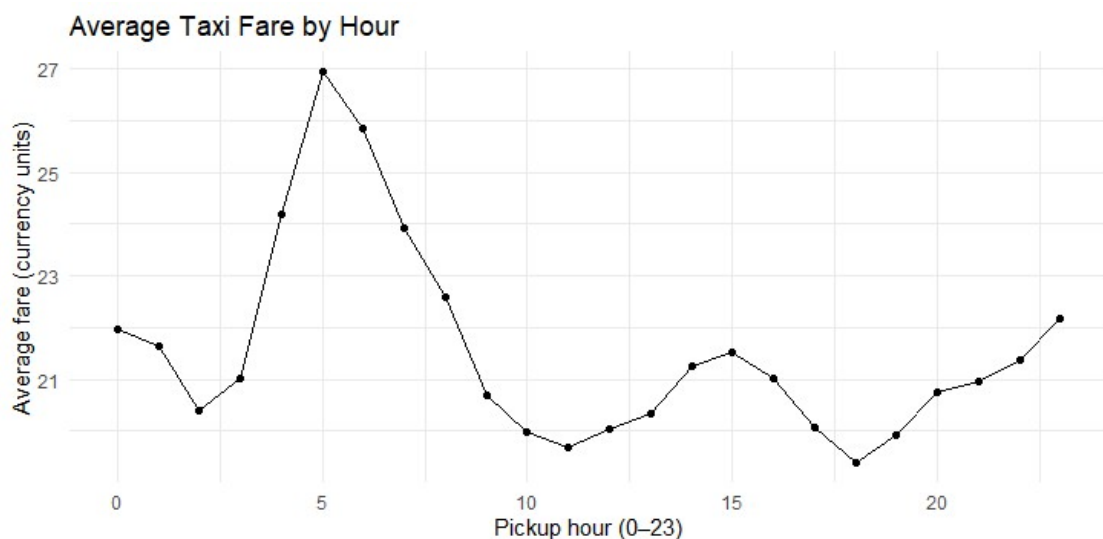
2. Taxi demand by weekday



3. Monthly Taxi demand



4. Average Fare by hour



Interpretation of Analytical Findings

1. Temporal Demand and Revenue Dynamics

- **Hourly Dynamics:** Taxi demand exhibits a classic urban peak pattern. The lowest activity occurs between 03:00 and 05:00 (dropping to ~27,740 trips at 04:00). Demand climbs steadily through the morning, peaking during evening rush hours between 17:00 and 18:00 (~232,874 trips).
- **Fare Inversion during Off-Peak Hours:** Interestingly, while trip volume is lowest between 04:00 and 06:00, the average fare peaks during this exact window (\$26.95 at 05:00 vs. \$19.58–\$20.05 during afternoon/evening peaks). This

indicates longer distance trips (e.g., early morning airport runs or outer-borough commutes) during early morning hours.

- **Weekly Trends:** Demand scales up through the workweek, starting lowest on Monday (~358,979 trips) and Sunday (~365,150 trips), and peaking on Saturday (~658,124 trips) and Thursday/Friday (~617k–622k trips), highlighting strong weekend nightlife and commuter activity.
- **Monthly Distribution:** The dataset primarily reflects January 2026 data (3,562,052 trips), with negligible stray entries in February (1 trip) and December (5 trips), indicating the scope is concentrated on winter monthly operations.

2. Spatial and Route Analysis

- **Borough Distribution:** Manhattan overwhelmingly dominates NYC taxi operations, accounting for 3,051,535 out of ~3.56M cleaned trips (~85.7%) and generating \$77.87M in revenue. Queens ranks second with 323,258 trips, but boasts a significantly higher average fare (\$48.57) due to airport travel.
- **Volume vs. Revenue Routes:**
 - *Top Volume Routes:* Highly concentrated short-haul intra-Manhattan corridors (e.g., Upper East Side South → Upper East Side North with 23,836 trips).
 - *Top Revenue Routes:* Long-distance airport transfers dominate total revenue. The JFK Airport → Outside of NYC route yielded \$879,258.50 across 7,146 trips (average \$123.04/trip), followed by JFK Airport → Times Sq/Theatre District (\$476,810.70 across 4,902 trips).

3. Payment Preferences & Distance-Fare Anomalies

- **Payment Preference:** Credit cards/electronic payments (Payment Type 1) represent the vast majority of volume (2,231,447 trips generating \$65.6M). Type 0 represents unrecorded/missing payment codes (995,312 trips), while cash (Type 2) accounts for 297,883 trips.
- **Correlation Anomaly:** The linear correlation between trip_distance and fare_amount returned an unexpectedly low value of 0.0076. While group-aggregated averages show the expected monotonic increase (0–1 mile avg fare \$9.56 up to 20+ miles avg fare \$90.87), extreme distance outliers in the uncleaned data (e.g., max trip distance recorded as 269,100 miles) severely skewed the raw global correlation coefficient.

Critical Performance Analysis

1. Vectorized & In-Memory Operations vs. Functional Approaches

- **Data Aggregation:** Standard functional approaches like `lapply()` or `purrr::map()` process items sequentially or via unoptimized list iterations, requiring explicit list splitting (`hour_groups`).
- **data.table Efficiency:** By utilizing internal C-level grouping, contiguous memory allocation, and key-based indexing, `data.table` performs group-by aggregations natively on the in-memory `data.table` object without intermediate copy overhead. On ~3.56 million rows, `data.table` aggregations execute almost instantaneously ($< 0.1s$).

2. Sequential vs. Parallel Processing Benchmark

- Sequential Execution Time: 0.09 seconds
- Parallel Execution Time (foreach + doParallel over 4 partitions): 12.08 seconds
- Speedup: $\frac{T_{seq}}{T_{par}} = \frac{0.09}{12.08} \approx 0.007$

Conclusion

This experiment highlights that for large-scale dataset analytics in R (~3.56M+ rows), algorithm selection and framework efficiency far outweigh raw multi-core execution. Highly optimized, C-backed single-threaded implementations (`data.table`) dramatically outperform naive parallel implementations by bypassing IPC and memory serialization bottlenecks. Understanding these trade-offs ensures both computational speed and actionable business intelligence for urban transit management.